

Homework 3

- ① Goal : To make as few stops as possible at gas stations from USC to Santa Monica.

Algorithm: Start with a full tank at USC. From your current position, drive to the farthest gas station you can reach on your current tank (within distance p). Refuel there and repeat this process until you can reach Santa Monica.

High-level solution

1. Initialize your current position at USC with a full tank.
2. While you cannot yet reach Santa Monica:
 - From your current position, look ahead at all stations within your remaining fuel range.
 - Choose the farthest reachable station, drive there and refuel to full.
3. Repeat until you reach or surpass the distance of Santa Monica.

Proof by mathematical induction

We will prove that the greedy solution is optimal. Let the steps chosen by our greedy algorithm be $g_1, g_2, \dots$ and the steps in an optimal solution be $o_1, o_2, \dots$. We will prove by induction that our greedy strategy always "stays ahead" meaning $g_i \geq o_i$ for all steps i .

1. Best case: For the first step ($i=1$), our greedy algorithm chooses the farthest possible station g_1 reachable from the start. Any other solution's first stop, o_1 , must also be reachable from the start. Therefore, g_1 must be at a distance greater than or equal to o_1 .
∴ The statement $g_1 \geq o_1$ is true.

2. Inductive hypothesis: We assume that $g_i \geq o_i$ is true for the i th stop.

3. Inductive step: We must prove it is also true for $i+1$ -th stop.

Our next stop, g_{i+1} , is the farthest station reachable from g_i .
Their next stop, o_{i+1} , is the farthest

station reachable from o_i .

Since our starting point g_i is faster than or equal to their starting point o_i , the maximum distance we can travel is also farther or equal to theirs.

$$g_{i+1} \geq o_{i+1}$$

Conclusion: Our greedy choice g_i is always at least as far as the optimal choice o_i for every stop i . Since our strategy makes at least as much progress with each stop, it cannot require more stops than the optimal solution, proving it is correct.

Time Complexity:

The time complexity is $O(n)$ where n is the number of gas stations. The algorithm makes a single pass through the list of stations. Over the whole trip we look at each gas station in the list at most once, without going back.

- ② To maximize the score by the formula $\prod_{i=1}^n a_i^{b_i}$, we need to arrange the sequences A and B such that the largest numbers are paired with the largest and so on in ~~the~~ ascending or order.

Algorithm:

1. Sort sequence A in ascending order.
2. Sort sequence B in ascending order.
3. Compute the maximum score as
$$\text{Score} = \prod_{i=1}^n A_{\text{sorted}}[i]^{B_{\text{sorted}}[i]}$$

Proof of correctness:

We can prove this algorithm is correct by using proof by contradiction.

1. Assume we have an optimal arrangement that is not perfectly sorted.
2. This means there must exist at least two pairs of elements, (a_i, b_i) and (a_j, b_j) , that are unsorted or out of order, such that in a sorted pair if $a_i < a_j$ and $b_i < b_j$ then in our case $a_i < a_j$ and $b_i > b_j$, their partners are swapped.

3. The contribution of these two pairs to the total score B , is $a_i b_i \times a_j b_j$

4. We swap the exponents to be (a_i, b_j) and (a_j, b_i) to give a new contribution:

$$a_i b_j \times a_j b_i$$

which is the optimal arrangement in our assumption.

Example: $2^4 \times 5^1 = 80$ ($a_i b_i \times a_j b_j$)

v.s.

$2^1 \times 5^4 = 1250$ ($a_i b_j \times a_j b_i$)

Clearly, the swapped version is larger where the pairs ~~have~~ are the largest in their sequence.

5. In general, for $a_j > a_i > 0$ and $b_i > b_j > 0$, the inequality

$$\left(\frac{a_j}{a_i}\right)^{b_i - b_j} > 1 \text{ is always true}$$

which proves that

$$a_i b_i \cdot a_j b_j < a_i b_j \cdot a_j b_i$$

Thus, any arrangement that isn't sorted can be improved by making a swap.

1. The only arrangement that cannot be improved is the one where both lists are sorted.

2. The sorted arrangement gives the maximum score.

Time Complexity:

1. Sorting A : $O(n \log n)$

2. Sorting B : $O(n \log n)$

3. Calculating the product : $O(n)$

$\therefore O(n \log n) + O(n \log n) + O(n) = O(n \log n)$.

The greedy algorithm of sorting both sequences and pairing them index by index produces the maximum score in $O(n \log n)$ time.

③ To minimize total calories when connecting two chains, we will always combine the two chains with the smallest available weights at each step.

This ensures that larger weights are processed as late as possible, minimizing their contribution to the total cumulative cost.

We will use min-heap data structure implementation to retrieve the two smallest chains at each step.

Algorithm:

1. Initialization:

- Create a min-heap and insert all n initial chain weights, $w_1, w_2, \dots, w_n$ to it.
- Initialize a variable TotalCost to 0.

2. Iterative Combination:

- While the min-heap contains more than one element:

- a. Take the two minimum weights from the heap; let them be w_a and w_b .
- b. Calculate the cost of combining these two chains:

$$\text{current_cost} = 100 * (w_a + w_b)$$
- c. Add this to the cumulative total:

$$\text{TotalCost} = \text{TotalCost} + \text{current_cost}$$
- d. Create the new combined chain's weight:

$$\text{new_weight} = w_a + w_b$$
- e. Insert new weight back into min-heap.

3. final Result:

- After $n-1$ iterations, the heap will contain a single element representing the total weight of the final chain.
- TotalCost value is the minimum possible calories burnt.

Time Complexity:

1. Building the initial min-heap from the n input weights takes $O(n)$ time.
- ~~2. Each of the $n-1$ operations of extracting and inserting take $O(\log n)$ time.~~
2. Each of the $n-1$ iterations of extracting and inserting heap operations take $O(\log n)$ time.
Hence it is $(n-1) \times O(\log n)$ time,
which is $O(n \log n)$ time.

Total time complexity is ,
 $O(n) + O(n \log n)$ which simplifies
to $O(n \log n)$

$\therefore O(n \log n)$ is asymptotically faster
than $O(n^2)$.